

Trabajo Práctico N°3

Redes Neuronales Convolucionales

Saccone Valentina, Terán Nougués María Belén

*Cátedra de Inteligencia Artificial
Ingeniería en Sistemas de Información*

Resumen

Palabras Claves: Redes Neuronales Convolucionales, Transfer Learning, Funciones de activación, Clasificación, Accuracy, Loss

1. Introducción

El presente trabajo aborda un problema de clasificación de imágenes por categorías basado en un dataset llamado "Intel Image Classification". El mismo fue acotado y está conformado por imágenes de edificios, bosques y glaciares, con un total de 7200 imágenes divididas en 3 clases.

Para cumplir con los objetivos propuestos, se codificaron tres redes neuronales convolucionales: una con arquitectura personalizada, otra con arquitectura VGG16 y la última con arquitectura VGG19. La codificación se llevó a cabo mediante la herramienta Google Colab, haciendo uso de TensorFlow y Keras.

Se realizan comparaciones de las tres arquitecturas planteadas para establecer cuál de ellas alcanza mayor exactitud y menor pérdida, y como en estas últimas dos afecta la utilización de Transfer Learning.

2. Propuesta de Desarrollo

2.1. Redes Neuronales Artificiales (RNA)

Las Redes de Neuronas Artificiales (RNA) son un modelo matemático simplificado e inspirado en cómo el sistema nervioso procesa la información. Funciona utilizando a la misma vez un número elevado de unidades de procesamiento interconectadas que parecen versiones abstractas de neuronas.

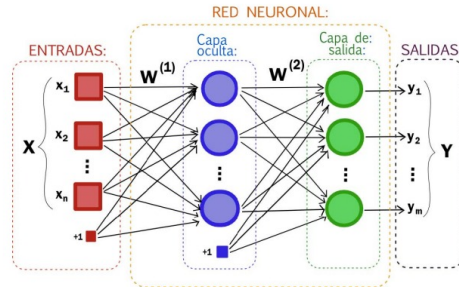


Figura 1 – Red Neuronal Artificial

La distribución de neuronas dentro de la red se realiza formando capas, con un número determinado de dichas neuronas en cada una de ellas. Se pueden distinguir tres tipos de capas:

1. Capa de entrada: es la capa que recibe directamente la información proveniente de las fuentes externas de la red.
2. Capas Ocultas: son internas a la red y no tienen contacto directo con el entorno exterior. El número de niveles ocultos puede estar entre cero y un número elevado. Las neuronas de las capas ocultas pueden estar interconectadas de distintas maneras, lo que determina, junto con su número, las distintas topologías de redes neuronales.
3. Capa de salida: transfiere información de la red hacia el exterior.

Se dice que una red está totalmente conectada si todas las salidas desde un nivel llegan a todos y cada uno de los nodos del siguiente nivel para resolver un problema determinado.[1]

Asimismo, se destaca la forma de conexión entre las neuronas. La utilizada en este trabajo es la red no recurrente llamada "Feedforward", donde la propagación de las señales se produce en solo un sentido, hacia adelante. Al contrario, las redes recurrentes poseen lazos de retroalimentación, que pueden ser entre neuronas de diferentes capas, de la misma capa o incluso de una neurona consigo misma.[2]

Además, el peso de una conexión gradúa la importancia de la conexión existente entre la neurona receptora y el emisor de la señal (generalmente otra neurona). El valor del peso es ajustable en función de la experiencia, lo que permite que las redes de neuronas artificiales sean instrumentos adaptables a varios tipos de entradas y tengan la capacidad de aprender.

2.2. Redes Neuronales Convolucionales (CNN)

Las Redes Neuronales Convolucionales (CNN) son redes neuronales artificiales con aprendizaje supervisado, que procesan sus capas imitando el cortex visual del ojo humano para identificar distintas características en la entrada. Esto logra que en definitiva puedan identificar objetos.

Los tres tipos de problemas que resuelven son:

- Detección/categorización de objetos.
- Segmentación.
- Clasificación de imágenes.[3]

Para ello, la CNN contiene varias capas ocultas especializadas y con una jerarquía, esto significa que las primeras capas detectan propiedades o formas básicas y se van especializando hasta llegar a capas más profundas capaces de reconocer formas complejas como un rostro o una silueta.

Para el problema de clasificación a abordar en el trabajo podemos dividir al conjunto de capas en parte de visión y parte de clasificación.

Destacamos dos tipos de capas para la parte de visión:

1. Capas Convolucionales: consisten en una malla rectangular de neuronas. Requieren que la capa anterior también sea una malla rectangular de neuronas. Cada neurona toma una entrada desde una sección rectangular de la capa anterior, los pesos se mantienen para cada neurona en la misma capa convolucional. Por ello, la capa convolucional es una convolución de imagen de la capa previa, donde los pesos especifican el filtro de convolución. Adicionalmente, pueden haber diversas mallas en cada capa convolucional; cada malla toma entradas desde todas las mallas en la capa anterior, usando diferentes filtros potenciales. Estas capas utilizan funciones de activación.
2. Capa de Pooling: Después de cada capa convolucional, podría haber una capa de pooling. Existen de dos tipos: MaxPooling y MeanPooling. Para el trabajo utilizamos capas de MaxPooling, las cuales reducen las dimensiones de las salidas de las capas convolucionales seleccionando el valor máximo en regiones de la imagen. Esto ayuda a disminuir el tamaño del mapa de características, reduciendo la carga computacional y controlando el sobreajuste. [4]

Luego de las capas mencionadas anteriormente, se recurre a una red neuronal multicapa, la cual conforma la parte de clasificación:

1. Capa Flatten: convierte las salidas multidimensionales de las capas convolucionales en un vector unidimensional.
2. Capas Densas: cada una de estas toma todas las salidas de las neuronas previas y las conecta a cada neurona que posee. Estas capas realizan la combinación lineal de las entradas y aplican una función de activación, siendo esenciales para las decisiones de clasificación y regresión en la red neuronal.
3. Dropout: desactiva un número determinado de neuronas de una capa de forma aleatoria. Las neuronas desactivadas no se tienen en cuenta para la propagación hacia delante ni para atrás, lo que obliga a las neuronas cercanas a trabajar mejor de forma solitaria y no depender tanto de las neuronas desactivadas. Esto ayuda a reducir el overfitting.[1]

2.3. Funciones de Activación

Como previamente se mencionó, las redes neuronales convolucionales utilizan funciones de activación. Las siguientes fueron utilizadas en el desarrollo del trabajo:

- ReLu (Rectified Linear Unit): es la función de activación más utilizada en los modelos de aprendizaje profundo. La función devuelve 0 si recibe una entrada negativa, pero para cualquier valor positivo devuelve ese valor. Por lo tanto, se puede escribir como $f(x) = \max(0, x)$. Introduce la no linealidad en el modelo, permite una convergencia más rápida durante el entrenamiento y ayuda a resolver el problema del desvanecimiento del gradiente que afecta a otras funciones de activación como sigmoide o tangente hiperbólica.
- Softmax (función exponencial normalizada): es una generalización de la función sigmoide. Se utiliza como función de activación de salida para la clasificación multiclase porque escala las entradas precedentes de un rango entre 0 y 1 y normaliza la capa de salida, de modo que la suma de todas las neuronas de salida sea igual a la unidad. Se considera a esta función como una distribución de probabilidad categórica, lo que le permite comunicar un grado de confianza en las predicciones de su clase. [2]

2.4. Transfer Learning

El aprendizaje por transferencia (Transfer Learning) es una de las técnicas más importantes del deep learning en el ámbito del aprendizaje automático en inteligencia artificial. Consiste en aprovechar una gran cantidad

de información adquirida durante la resolución de un problema y utilizarla para abordar otro problema diferente, siempre y cuando ambos compartan características similares. En otras palabras, se trata de modificar patrones ya entrenados (o redes neuronales) para reconocer ciertas características y así poder reconocer otras similares sin necesidad de realizar el entrenamiento desde cero en cada ocasión.

Se destaca como un recurso que ahorra tiempo y reduce significativamente la cantidad de cálculos necesarios. Con los datos obtenidos de una tarea similar, pero más genérica, evita la necesidad de realizar aproximaciones costosas para llegar a un punto de partida óptimo[8].

2.5. Contexto del Problema

El trabajo se basa en la resolución de un problema de clasificación por categorías, utilizando un dataset denominado "Intel Image Classification" obtenido de la página de *Kaggle*.¹ Este dataset cuenta originalmente con 27335 imágenes de tamaño 150x150 píxeles, conformadas por 6 categorías con las siguientes etiquetas:

0. Buildings.
1. Forest.
2. Glacier.
3. Mountain.
4. Sea.
5. Street.

Por limitaciones de las herramientas utilizadas, las categorías fueron reducidas a tres, con 2400 imágenes cada una, llegando a un total de 7200 imágenes en total. Las categorías elegidas poseen las siguientes etiquetas:

0. Buildings.
1. Forest.
2. Glacier.

Es así que el dataset fue renombrado a "landscapes.dataset", donde establecimos que para cada una de las categorías hay 2000 imágenes para entrenamiento (train), 200 para validación (valid) y 200 para evaluación (test).

¹<https://www.kaggle.com/datasets/puneet6060/intel-image-classification?resource=download>

Para la codificación utilizamos la herramienta *Google Colab* ², un servicio alojado de *Jupyter Notebook* ³ que no requiere configuración y que ofrece acceso a recursos de computación limitados en su versión gratuita.

También utilizamos *TensorFlow* ⁴, un software de computación numérica creado por Google, orientado a problemas de Deep Learning. En el proyecto se demuestra la utilidad que presenta para construir y entrenar redes neuronales. TensorFlow es una librería desarrollada a partir de la combinación de C++ y CUDA. La principal estructura de datos que se maneja en esta librería son los tensors. Con un tensor nos referimos a un conjunto de valores primitivos, por ejemplo números enteros o flotantes, organizados por un array de 1 o N dimensiones, donde el rango del tensor sería el número de dimensiones.

Posee integraciones con otras bibliotecas del ecosistema como *Keras* ⁵, una API de redes neuronales a alto nivel, escrita en Python. Fue desarrollada con el enfoque de permitir una experimentación rápida, es decir, pasar de la idea al resultado con el menor retraso posible.

La codificación se realiza con *Python* ⁶, un lenguaje de programación de alto nivel, interpretado y de propósito general, conocido por su simplicidad y legibilidad. Permite a los desarrolladores implementar rápidamente soluciones eficientes y robustas. Además, es multiplataforma, lo que significa que se puede ejecutar en diferentes sistemas operativos como Windows, macOS y Linux.

NumPy ⁷, es el paquete fundamental para la computación científica en Python. Es una biblioteca que proporciona un objeto de matriz multidimensional, varios objetos derivados (como matrices y matrices enmascaradas) y una variedad de rutinas para operaciones rápidas en matrices, incluidas operaciones matemáticas, lógicas, manipulación de formas, ordenamiento, selección, E/S, transformadas de Fourier discretas, álgebra lineal básica, operaciones estadísticas básicas, simulación aleatoria y mucho más.

También se utilizó *Matplotlib* ⁸, una biblioteca de Python para la generación de gráficos en dos dimensiones.

²<https://colab.google/>

³<https://jupyter.org/>

⁴<https://www.tensorflow.org/?hl=es-419>

⁵<https://keras.io/>

⁶<https://www.python.org/>

⁷<https://numpy.org/doc/stable/>

⁸<https://matplotlib.org/>

2.6. Preprocesamiento de imágenes:

Para el preprocesamiento de imágenes es necesario realizar un proceso de Data Augmentation (aumento de datos) debido al tamaño de las imágenes. Se trata de una técnica que consiste en variar la forma, tamaño, posición de una imagen para que la red neuronal reciba una mayor cantidad de información. [2]

Asimismo, es necesario convertir los arrays de las etiquetas en one hot encoding. One hot encoding es una técnica de preprocesamiento comúnmente utilizada en el aprendizaje automático y en las redes neuronales para representar etiquetas categóricas como vectores binarios. Esta técnica es especialmente útil cuando se trabaja con algoritmos que no pueden manejar etiquetas categóricas directamente. En términos simples, one hot encoding transforma cada categoría en un vector en el que solo un elemento es 1 (indicando la presencia de esa categoría) y todos los demás elementos son 0. Esto ayuda a las redes neuronales a entender y procesar las etiquetas categóricas de manera eficiente.

2.7. Modelo

Se plantea el modelo de la red neuronal convolucional, que incluye una parte de visión y una de clasificación. También se especifica mediante variables el tamaño del batch size y la cantidad de épocas.

- Batch Size: es la cantidad de muestras de entrenamiento utilizadas en una iteración.
- Epochs: se trata de un ciclo de entrenamiento completo que abarca todas las muestras del conjunto de datos de entrenamiento. La cantidad de épocas determina cuántas veces el modelo verá todos los datos de entrenamiento antes de completar el entrenamiento.

2.7.1. Vision

La parte de visión es distinta para los tres casos.

Modelo con Arquitectura Personalizada

El modelo personalizado posee dos capas convolucionales detalladas a continuación:

Capa de entrada:

- Capa Conv2D con 32 neuronas, kernel de 3x3 y función de activación Relu.

- MaxPooling2D con tamaño de ventana de 2x2.
- Dropout de 10 %.

Segunda capa convolucional:

- Capa Conv2D con 64 neuronas, kernel de 3x3 y función de activación Relu.
- MaxPooling2D con tamaño de ventana de 2x2.
- Dropout de 10 %.

Modelo con Arquitectura VGG16

La arquitectura VGG16 está constituida por 13 capas convolucionales como se puede ver en Figura 2. Cada grupo es seguido de una capa de Max Pooling. A estas le siguen 3 capas densas totalmente conectadas, de ahí que su nombre incluya 16. Finalmente posee un clasificador softmax. Esta red tiene la ventaja de contar con pesos preentrenados disponibles del dataset de Imagenet, que podrían ser utilizados para inicializar el entrenamiento de la red propuesta.[5]

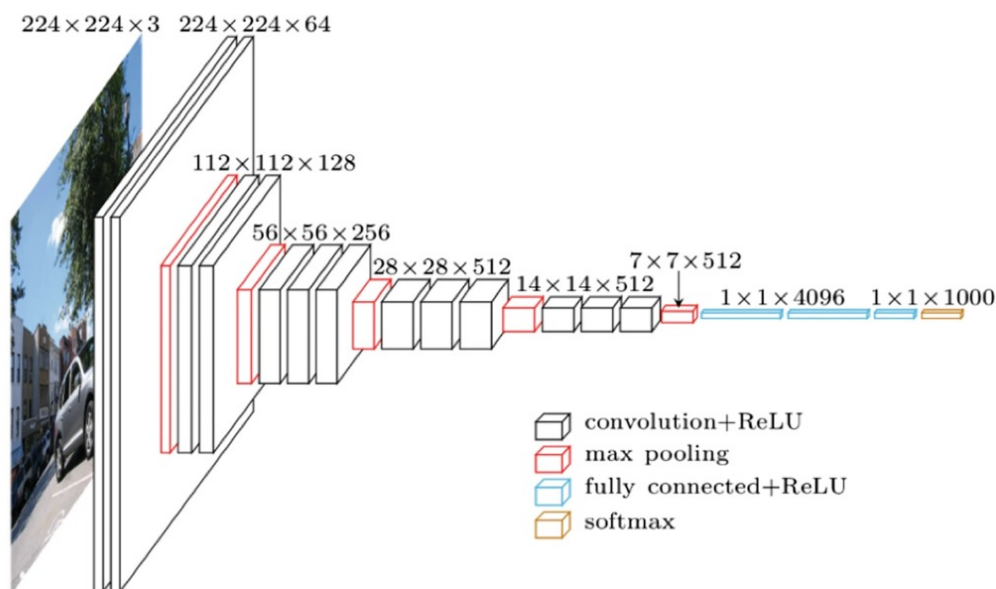


Figura 2 – Arquitectura de Red Neuronal Convolucional VGG16

En la codificación se importa el modelo VGG16 y se utiliza la función `preprocess_input` con los arrays `train` y `test`, lo cual ajusta el formato de las imágenes para que coincidan con el de las utilizadas en el preentrenamiento.

Como modelo base se crea una instancia del modelo VGG16. Allí, se utiliza `transfer learning` con imágenes del dataset *imagenet*⁹, con lo cual se mantienen los pesos logrados. Cabe destacar que solo se utilizan las capas convolucionales pertenecientes a la parte de visión, especificando esto en código como `include_top = False`.

Seguidamente se congela este modelo base con `trainable = False`, para que de este modo los pesos no se actualicen en el posterior entrenamiento.

Modelo con Arquitectura VGG19

La arquitectura de VGG-19 es muy similar a la de la VGG16. La diferencia principal de VGG19 radica en el número de capas, agregando hasta 19 capas para aprender los pesos, tal y como se muestra en Figura 3 [6]

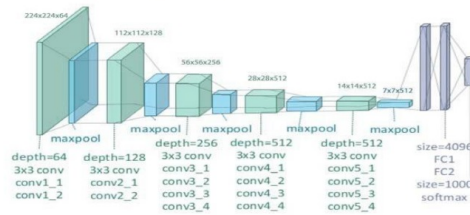


Figura 3 – Arquitectura de Red Neuronal Convolutional VGG19

En la codificación se importa el modelo VGG19 y se utiliza la función `preprocess_input` con los arrays `train` y `test`, lo cual ajusta el formato de las imágenes para que coincidan con el de las utilizadas en el preentrenamiento.

Como modelo base se crea una instancia del modelo VGG19. Allí, se utiliza `transfer learning` de la misma manera que con el modelo de arquitectura VGG16. Asimismo, también ocurre que solo se utilizan las capas convolucionales pertenecientes a la parte de visión, especificando esto en código como `include_top = False`.

Nuevamente se congela el modelo base con `trainable = False`, para que los pesos no se actualicen en el posterior entrenamiento.

⁹<https://www.image-net.org/>

2.7.2. Clasificación

La parte de clasificación es la misma para todas las arquitecturas planteadas, con la finalidad de realizar una comparación. Esta comienza con una capa Flatten, seguida de una capa densa de 50 neuronas con función de activación relu y dropout con parámetro 20 %. A continuación, una capa densa de 20 neuronas con función de activación relu y dropout con parámetro de 20 %. Finalmente, la capa de salida es una capa densa de 3 neuronas correspondientes a las 3 clases del problema, y se utiliza la función softmax.

2.8. Compilación y Entrenamiento

Para la compilación del modelo se utiliza la función compile. La misma posee los siguientes parámetros:

- `loss = categorical_crossentropy`. La función de pérdida categorical crossentropy evalúa la diferencia entre las probabilidades predichas por el modelo y las etiquetas reales. Penaliza más las predicciones incorrectas, lo que ayuda al modelo a aprender mejor durante el entrenamiento.
- `optimizer = Adam`. El optimizador Adam (Adaptive Moment Estimation) combina las ventajas de otros dos métodos (AdaGrad y RMS-Prop), ajustando las tasas de aprendizaje para cada parámetro de forma adaptativa. Es eficiente y requiere poca memoria, lo que lo hace ideal para muchos problemas de aprendizaje profundo.
- `metrics = ['accuracy']`. La métrica accuracy mide la precisión del modelo, es decir, la proporción de predicciones correctas sobre el total de predicciones realizadas.

Para el entrenamiento del modelo compilado se utiliza en primer lugar la función EarlyStopping con los siguientes parámetros:

- `Monitor`. Se indica el parámetro a tener en cuenta para detener el entrenamiento.
- `Mode`. Se define cómo se debe interpretar la métrica de evaluación para decidir cuándo detener el entrenamiento.
- `Patience`. Se indica el número de épocas luego de las cuales el entrenamiento se detiene al no presentar mejora.

- `Restore_best_weights`. Determina si se deben restaurar los pesos del modelo al valor que tenían en el punto en que la métrica de evaluación fue mejor

Luego, se utiliza la función `fit` que posee los parámetros definidos a continuación:

- `Los datos de entrenamiento`.
- `Batch_size`. Tamaño de batch.
- `Epochs`. Cantidad de épocas.
- `Validation_data`. Los datos utilizados para validación
- `callbacks = [es]`. Así se indica el llamado a la función `EarlyStopping` detallada anteriormente.

2.9. Evaluación y Métricas

Para la evaluación del modelo entrenado, se utiliza la función `evaluate` con los arrays correspondientes a test. Esto da como resultado `test loss` y `test accuracy`, que corresponden a números de pérdida y exactitud arrojados por el modelo con las imágenes de evaluación.

Seguidamente se muestran las gráficas resultantes del entrenamiento: "Training and Validation Accuracy" donde se ve el proceso de la exactitud que va alcanzando el modelo en entrenamiento y validación y "Training and Validation Loss" que muestra el proceso de pérdida que obtiene el modelo en entrenamiento y validación.

Se muestra una tabla con la función "classification_report", la cual posee columnas a destacar:

- **Precision**: Es una medida que indica que proporción de verdaderos positivos hay entre los predichos de cada clase (verdaderos más falsos positivos).

$$Precision = \frac{VerdaderosPositivos(TP)}{VerdaderosPositivos(TP) + FalsosPositivos(FP)}$$

- **Recall**: Mide la capacidad del modelo para identificar correctamente todas las instancias positivas en un conjunto de datos. Es decir, indica qué proporción de los casos positivos reales de cada clase fueron correctamente identificados por el modelo.

$$Recall = \frac{VerdaderosPositivos(TP)}{VerdaderosPositivos(TP) + FalsosNegativos(FN)}$$

- F1 score: El puntaje F1 es la media armónica de la precisión y el recall. Es una métrica combinada que proporciona un balance entre precision y recall.

$$F1score = \frac{2xy}{x+y}$$

La media armónica es una especie de promedio cuando x e y son iguales. Pero cuando son diferentes, entonces está más cerca del número más pequeño en comparación con el número más grande.[2]

- Support: Establece el número de casos reales de cada clase en el conjunto de imágenes. Es la cantidad de instancias que efectivamente pertenecen a cada clase.

Además, en la tabla también se destacan tres filas:

- Accuracy: Representa la exactitud general del modelo, la proporción de todas las predicciones correctas (verdaderos positivos + verdaderos negativos) entre el número total de instancias.
- Macro Avg: Es la media aritmética de la precisión, recall y F1-score de todas las clases.
- Weighted Avg: Se trata de la media ponderada de la precisión, recall y F1-score de todas las clases.

Finalmente, se utiliza la Matriz de Confusión para una evaluación final. La misma proporciona información útil sobre las etiquetas de imágenes reales y las etiquetas de imágenes predichas propuestas por el método de clasificación. Se pueden evaluar diferentes aspectos del desempeño de la clasificación utilizando esta información.

La matriz muestra tres valores a destacar:

- Los valores en la diagonal muestran los verdaderos positivos (TP), es decir, el número de muestras que se clasifican como verdadera cuando en realidad lo son.

- Los valores por debajo de la diagonal son Falsos Negativos (FN), el número de muestras que se clasifican como negativas cuando en realidad lo son.
- Los valores por encima de la diagonal son Falsos Positivos (FP), el número de muestras que se clasifican como verdaderas cuando son falsas. [2][7]

3. Experimentos y Resultados

3.1. Carga y preprocesamiento de imágenes:

Para la carga de imágenes se importa el dataset que previamente se cargó en *Google Drive* ¹⁰

Se descargan las imágenes desde las carpetas train, test y valid en tres arrays llamados train_ds, test_ds y valid_ds respectivamente. Luego, estos se convierten en arrays Numpy. Seguidamente se utiliza la función "Reshape" para asegurar que los arrays tienen la forma deseada, en este caso "reshape(-1, 150, 150, 3)". El número tres al final de la función indica los tres canales de colores de las imágenes.

A continuación, se procede a contar las etiquetas de cada clase en los tres conjuntos, para mostrar que efectivamente el conjunto train está conformado por 2000 imágenes de cada clase y los conjuntos test y valid están compuestos por 200 imágenes de cada clase. Con esto se realizan gráficos de barras, demostrando de forma visual lo previamente establecido.

¹⁰https://www.google.com/intl/es_ar/drive/



Figura 4 – Gráfico de barras del conjunto de imágenes de train. Muestra la cantidad de imágenes perteneciente a cada una de las tres clases.

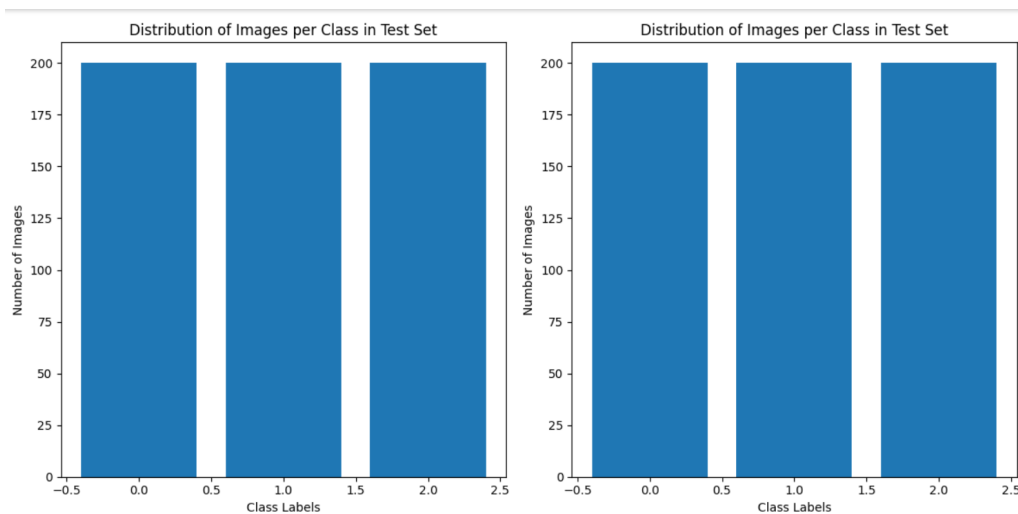


Figura 5 – Gráficos de barrass de los conjuntos de imágenes de test y valid. Muestra la cantidad de imágenes perteneciente a cada una de las tres clases en cada conjunto.

Se utiliza la función "Resize" para cambiar el tamaño de las imágenes de 150x150 píxeles a 224x224 píxeles (Data Augmentation). Esto se debe a que las arquitecturas elegidas están pre-entrenadas con imágenes grandes, por lo cual es necesario adecuar el tamaño de las pertenecientes al dataset elegido. Lo que logra esta función es que las imágenes se estiren hasta llegar al tamaño indicado.

Se muestran las imágenes luego de realizado este proceso:



Figura 6 – Imágenes de muestra de cada clase del dataset.

Finalmente, se utiliza la función "to_categorical" para convertir los arrays de las etiquetas en one hot encoding.

En este caso como tenemos tres categorías: Buildings (0), Forest (1) y Glacier (2), one hot encoding las convierte de la siguiente manera:

- Buildings (0): [1 0 0]
- Forest (1): [0 1 0]
- Glacier (2): [0 0 1]

3.2. Modelo con Arquitectura Personalizada

Al compilar el modelo con arquitectura personalizada, podemos observar detalladamente las capas que el mismo posee, la forma de la salida y los parámetros de cada una de estas.

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 111, 111, 32)	0
dropout_5 (Dropout)	(None, 111, 111, 32)	0
conv2d_4 (Conv2D)	(None, 109, 109, 64)	18496
max_pooling2d_4 (MaxPooling2D)	(None, 54, 54, 64)	0
dropout_6 (Dropout)	(None, 54, 54, 64)	0
flatten_1 (Flatten)	(None, 186624)	0
dense_3 (Dense)	(None, 50)	9331250
dropout_7 (Dropout)	(None, 50)	0
dense_4 (Dense)	(None, 20)	1020
dropout_8 (Dropout)	(None, 20)	0
dense_5 (Dense)	(None, 3)	63

Figura 7 – Imagen del modelo con arquitectura personalizada luego de compilado. Se describen cada una de sus capas.

Seguidamente, se visualizan la cantidad total de parámetros y cuántos de estos son y no entrenables.

```
Total params: 9351725 (35.67 MB)
Trainable params: 9351725 (35.67 MB)
Non-trainable params: 0 (0.00 Byte)
```

Figura 8 – Imagen de los parámetros del modelo de arquitectura personalizada. Se detallan total de parámetros, y cantidades entrenables y no entrenables

Para el entrenamiento se eligió un batch size de 16, y 25 épocas. El mismo se realiza mediante las imágenes de train (train_X y train_Y), y se valida utilizando valid (valid_X y valid_Y). Durante el proceso, se detalla la exactitud y pérdida en entrenamiento y en validación.

El algoritmo de Early Stopping con patience = 5 logra que luego de 5 valores bajos luego del más alto detectado de val_accuracy el entrenamiento finalice. Es así, que el entrenamiento llegó a las 18 épocas:


```

Epoch 1/25
375/375 [=====] - 18s 45ms/step - loss: 11.5586 - accuracy: 0.6562 - val_loss: 0.7098 - val_accuracy: 0.7311
Epoch 2/25
375/375 [=====] - 16s 42ms/step - loss: 0.7071 - accuracy: 0.7248 - val_loss: 0.8310 - val_accuracy: 0.6883
Epoch 3/25
375/375 [=====] - 16s 43ms/step - loss: 0.5077 - accuracy: 0.8150 - val_loss: 0.6175 - val_accuracy: 0.7833
Epoch 4/25
375/375 [=====] - 16s 43ms/step - loss: 0.4270 - accuracy: 0.8465 - val_loss: 0.6731 - val_accuracy: 0.7817
Epoch 5/25
375/375 [=====] - 16s 42ms/step - loss: 0.4367 - accuracy: 0.8467 - val_loss: 0.8638 - val_accuracy: 0.7050
Epoch 6/25
375/375 [=====] - 16s 43ms/step - loss: 0.2947 - accuracy: 0.9002 - val_loss: 0.6825 - val_accuracy: 0.8217
Epoch 7/25
375/375 [=====] - 17s 45ms/step - loss: 0.2679 - accuracy: 0.9117 - val_loss: 0.6647 - val_accuracy: 0.8083
Epoch 8/25
375/375 [=====] - 21s 55ms/step - loss: 0.2334 - accuracy: 0.9285 - val_loss: 0.6986 - val_accuracy: 0.8217
Epoch 9/25
375/375 [=====] - 16s 44ms/step - loss: 0.1926 - accuracy: 0.9408 - val_loss: 0.7605 - val_accuracy: 0.7933
Epoch 10/25
375/375 [=====] - 16s 42ms/step - loss: 0.1960 - accuracy: 0.9490 - val_loss: 0.5406 - val_accuracy: 0.8583
Epoch 11/25
375/375 [=====] - 15s 41ms/step - loss: 0.2178 - accuracy: 0.9418 - val_loss: 0.6291 - val_accuracy: 0.8250
Epoch 12/25
375/375 [=====] - 16s 42ms/step - loss: 0.1331 - accuracy: 0.9593 - val_loss: 0.8104 - val_accuracy: 0.8350
Epoch 13/25
375/375 [=====] - 16s 41ms/step - loss: 0.1355 - accuracy: 0.9632 - val_loss: 0.6241 - val_accuracy: 0.8633
Epoch 14/25
375/375 [=====] - 15s 41ms/step - loss: 0.1497 - accuracy: 0.9615 - val_loss: 0.8031 - val_accuracy: 0.8300
Epoch 15/25
375/375 [=====] - 16s 43ms/step - loss: 0.1530 - accuracy: 0.9630 - val_loss: 0.8926 - val_accuracy: 0.7967
Epoch 16/25
375/375 [=====] - 16s 42ms/step - loss: 0.1625 - accuracy: 0.9657 - val_loss: 0.9584 - val_accuracy: 0.7833
Epoch 17/25
375/375 [=====] - 16s 44ms/step - loss: 0.1421 - accuracy: 0.9625 - val_loss: 0.8220 - val_accuracy: 0.8217
Epoch 18/25
375/375 [=====] - 16s 42ms/step - loss: 0.0873 - accuracy: 0.9798 - val_loss: 0.6771 - val_accuracy: 0.8533

```

Figura 9 – Imagen del entrenamiento de la CNN con arquitectura personalizada.

Luego de este entrenamiento se realizó la evaluación del modelo, esto se logra mediante evaluar utilizando test (test_X y test_Y). Se llegó a un accuracy de 89% y loss de 0.4737

```

19/19 [=====] - 1s 34ms/step - loss: 0.4737 - accuracy: 0.8900
Test loss: 0.4736870229244232
Test accuracy: 0.8899999856948853

```

Figura 10 – Evaluación de la CNN con arquitectura personalizada luego de su entrenamiento.

Seguidamente, se realizaron gráficas para mostrar de manera visual el proceso de entrenamiento.

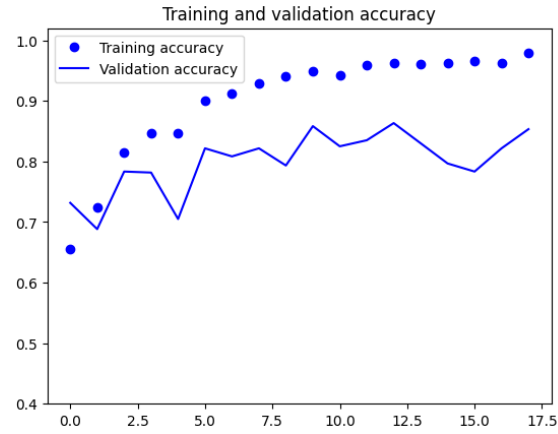


Figura 11 – Gráfico de exactitud en el proceso de entrenamiento y validación de CNN con arquitectura personalizada (training and validation accuracy).

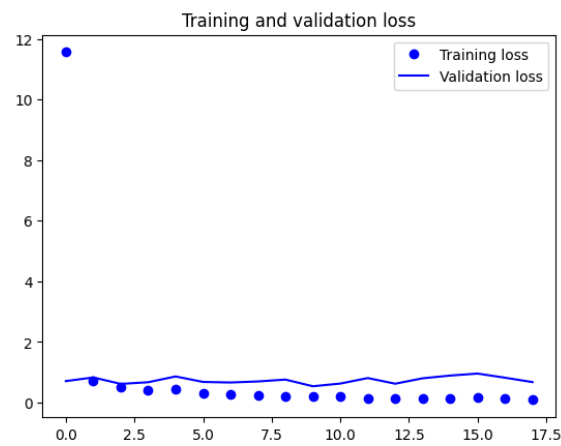


Figura 12 – Gráfico de pérdida en el proceso de entrenamiento y validación de CNN con arquitectura personalizada (training and validation loss).

Se realizó una tabla con `classification_report`. Se detallan en las columnas precision, recall y f1-score de cada una de las tres clases. Asimismo, se destaca nuevamente la exactitud de 89% del modelo, así como macro y weighted avg con los mismos valores para cada uno de los parámetros de las columnas previamente mencionadas.

	precision	recall	f1-score	support
Class 0	0.84	0.85	0.85	200
Class 1	0.92	0.88	0.90	200
Class 2	0.90	0.92	0.91	200
accuracy			0.89	600
macro avg	0.89	0.88	0.89	600
weighted avg	0.89	0.89	0.89	600

Figura 13 – Tabla de classification_report de CNN con arquitectura personalizada.

Finalmente, para completar la evaluación se realizó una matriz de confusión.

El análisis de esta matriz indica que las imágenes de edificios, bosques y glaciares se logran predecir hasta un cierto punto, siendo que de 200 muestras se predicen bien 170, 177 y 187 respectivamente. Esto reafirma la precisión de 89 % del modelo con arquitectura personalizada.

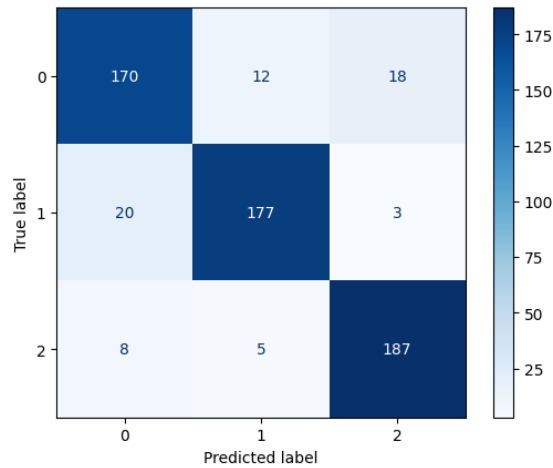


Figura 14 – Matriz de Confusión de CNN con arquitectura personalizada.

3.3. Modelo con Arquitectura VGG16

Luego de importar el modelo VGG16 utilizando transfer learning, se detallan las capas de visión que el mismo posee, la forma de la salida y los parámetros de cada una de estas.

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 200, 200, 3)]	0
block1_conv1 (Conv2D)	(None, 200, 200, 64)	1792
block1_conv2 (Conv2D)	(None, 200, 200, 64)	36928
block1_pool (MaxPooling2D)	(None, 100, 100, 64)	0
block2_conv1 (Conv2D)	(None, 100, 100, 128)	73856
block2_conv2 (Conv2D)	(None, 100, 100, 128)	147584
block2_pool (MaxPooling2D)	(None, 50, 50, 128)	0
block3_conv1 (Conv2D)	(None, 50, 50, 256)	295168
block3_conv2 (Conv2D)	(None, 50, 50, 256)	590080
block3_conv3 (Conv2D)	(None, 50, 50, 256)	590080
block3_pool (MaxPooling2D)	(None, 25, 25, 256)	0
block4_conv1 (Conv2D)	(None, 25, 25, 512)	1180160
block4_conv2 (Conv2D)	(None, 25, 25, 512)	2359808
block4_conv3 (Conv2D)	(None, 25, 25, 512)	2359808
block4_pool (MaxPooling2D)	(None, 12, 12, 512)	0

Figura 15 – Imagen de la parte de visión del modelo con arquitectura VGG16. Se describen cada una de sus capas. (Parte 1)

block5_conv1 (Conv2D)	(None, 12, 12, 512)	2359808
block5_conv2 (Conv2D)	(None, 12, 12, 512)	2359808
block5_conv3 (Conv2D)	(None, 12, 12, 512)	2359808
block5_pool (MaxPooling2D)	(None, 6, 6, 512)	0

Figura 16 – Imagen de la parte de visión del modelo con arquitectura VGG16. Se describen cada una de sus capas. (Parte 2)

Se visualizan la cantidad total de parámetros no entrenables, dado que se requiere mantener los pesos para solo entrenar la parte de clasificación.

```
Total params: 14714688 (56.13 MB)
Trainable params: 0 (0.00 Byte)
Non-trainable params: 14714688 (56.13 MB)
```

Figura 17 – Imagen de los parámetros de la parte de visión del modelo de arquitectura VGG16.

Luego se cargó la parte de clasificación, la cual se detalla en la siguiente imagen, con cada una de sus capas, forma de salida y parámetros.

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 7, 7, 512)	14714688
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 50)	1254450
dropout (Dropout)	(None, 50)	0
dense_1 (Dense)	(None, 20)	1020
dropout_1 (Dropout)	(None, 20)	0
dense_2 (Dense)	(None, 3)	63

Figura 18 – Imagen de la parte de clasificación del modelo con arquitectura VGG16. Se describen cada una de sus capas.

Se visualizan la cantidad total de parámetros de visión y clasificación, detallando la cantidad de aquellos entrenables correspondientes a clasificación y los no entrenables pertenecientes a visión.

```
Total params: 15970221 (60.92 MB)
Trainable params: 1255533 (4.79 MB)
Non-trainable params: 14714688 (56.13 MB)
```

Figura 19 – Imagen de los parámetros de la parte de clasificación del modelo de arquitectura VGG16.

Para el entrenamiento se eligió nuevamente un batch size de 16, y 25 épocas. También se realiza mediante las imágenes de train (train_X y train_Y), y se valida utilizando valid (valid_X y valid_Y). Durante el proceso, se detalla la exactitud y pérdida en entrenamiento y en validación.

El algoritmo de Early Stopping con patience = 5 logra que luego de 5 valores bajos luego del más alto detectado de val_accuracy el entrenamiento finalice. Es así, que el entrenamiento llegó a las 8 épocas:

```

Epoch 1/25
375/375 [=====] - 34s 85ms/step - loss: 0.0491 - accuracy: 0.9968 - val_loss: 1.4628 - val_accuracy: 0.9833
Epoch 2/25
375/375 [=====] - 29s 77ms/step - loss: 0.0553 - accuracy: 0.9973 - val_loss: 0.7578 - val_accuracy: 0.9883
Epoch 3/25
375/375 [=====] - 29s 78ms/step - loss: 0.0379 - accuracy: 0.9975 - val_loss: 0.3759 - val_accuracy: 0.9917
Epoch 4/25
375/375 [=====] - 32s 85ms/step - loss: 0.0205 - accuracy: 0.9988 - val_loss: 0.8544 - val_accuracy: 0.9917
Epoch 5/25
375/375 [=====] - 29s 78ms/step - loss: 0.0152 - accuracy: 0.9983 - val_loss: 1.0504 - val_accuracy: 0.9900
Epoch 6/25
375/375 [=====] - 29s 76ms/step - loss: 0.0256 - accuracy: 0.9982 - val_loss: 0.9966 - val_accuracy: 0.9900
Epoch 7/25
375/375 [=====] - 29s 77ms/step - loss: 0.0427 - accuracy: 0.9992 - val_loss: 1.2066 - val_accuracy: 0.9850
Epoch 8/25
375/375 [=====] - 29s 78ms/step - loss: 0.0106 - accuracy: 0.9995 - val_loss: 1.3221 - val_accuracy: 0.9900

```

Figura 20 – Imagen del entrenamiento de la CNN con arquitectura VGG16.

Luego de este entrenamiento se realizó la evaluación del modelo, esto se logra mediante evaluate utilizando al igual que antes test (test_X y test_Y). Se llegó a un accuracy de 99 % y loss de 0.052.

```

19/19 [=====] - 2s 127ms/step - loss: 0.0525 - accuracy: 0.9967
Test loss: 0.05254300311207771
Test accuracy: 0.9966666669845581

```

Figura 21 – Evaluación de la CNN con arquitectura VGG16 luego de su entrenamiento.

Seguidamente, se realizaron gráficas para mostrar de manera visual el proceso de entrenamiento.

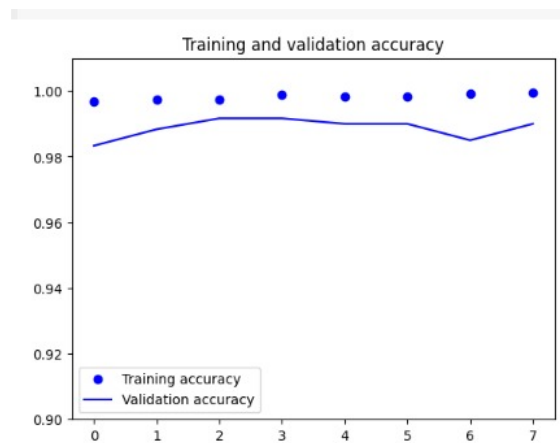


Figura 22 – Gráfico de exactitud en el proceso de entrenamiento y validación de CNN con arquitectura VGG16 (training and validation accuracy).

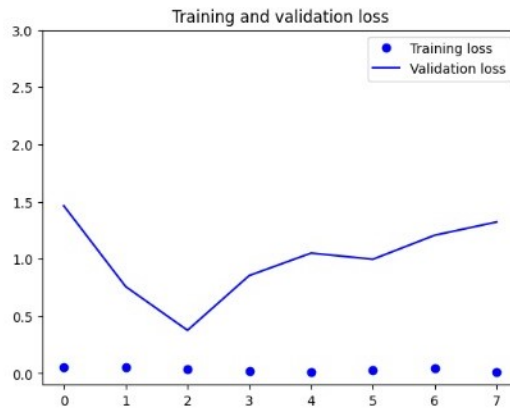


Figura 23 – Gráfico de pérdida en el proceso de entrenamiento y validación de CNN con arquitectura VGG16 (training and validation loss).

Se realizó una tabla con `classification_report`. Se detallan en las columnas precision, recall y f1-score de cada una de las tres clases. Asimismo, se destaca nuevamente la exactitud redondeada de 100 % del modelo, así como macro y weighted avg con los mismos valores para cada uno de los parámetros de las columnas previamente mencionadas.

	precision	recall	f1-score	support
Class 0	1.00	0.99	1.00	200
Class 1	0.99	1.00	1.00	200
Class 2	1.00	0.99	1.00	200
accuracy			1.00	600
macro avg	1.00	1.00	1.00	600
weighted avg	1.00	1.00	1.00	600

Figura 24 – Tabla de `classification_report` de CNN con arquitectura VGG16.

Finalmente se realizó una Matriz de Confusión. El análisis de esta indica que las imágenes de edificios, bosques y glaciares se predicen con alta precisión. Esto se denota en la diagonal de la matriz, donde los números son 200 o muy cercanos a este, mostrando que acierta la mayor parte de los datos. Así, queda demostrando que el modelo VGG16 clasifica correctamente para las categorías. En definitiva, la diagonal en la matriz de confusión reafirma la capacidad del modelo para distinguir eficazmente entre las distintas imágenes.

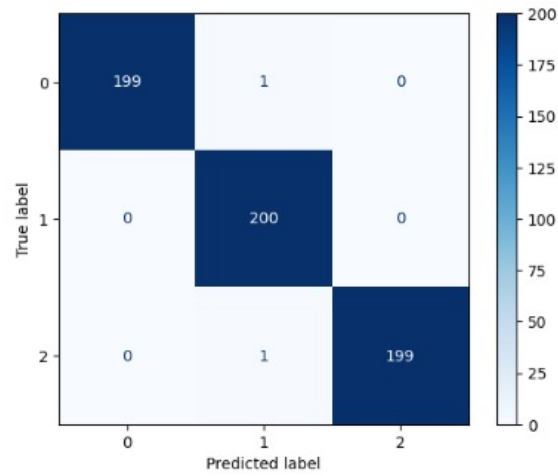


Figura 25 – Matriz de confusión VGG16.

3.4. Modelo con Arquitectura VGG19

Luego de importar el modelo VGG19 utilizando transfer learning, se detallan las capas de visión que el mismo posee, la forma de la salida y los parámetros de cada una de estas.

Model: "vgg19"		
Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 224, 224, 3)]	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590080
block3_conv4 (Conv2D)	(None, 56, 56, 256)	590080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1180160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2359808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2359808

Figura 26 – Imagen de la parte de visión del modelo con arquitectura VGG19. Se describen cada una de sus capas. (Parte 1)

block4_conv4 (Conv2D)	(None, 28, 28, 512)	2359808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv4 (Conv2D)	(None, 14, 14, 512)	2359808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0

Figura 27 – Imagen de la parte de visión del modelo con arquitectura VGG19. Se describen cada una de sus capas. (Parte 2)

Se visualizan la cantidad total de parámetros no entrenables, dado que se requiere mantener los pesos para solo entrenar la parte de clasificación.

```

=====
Total params: 20024384 (76.39 MB)
Trainable params: 0 (0.00 Byte)
Non-trainable params: 20024384 (76.39 MB)
=====

```

Figura 28 – Imagen de los parámetros de la parte de visión del modelo de arquitectura VGG19.

Luego se cargó la parte de clasificación, la cual se detalla en la siguiente imagen, con cada una de sus capas, forma de salida y parámetros.

Layer (type)	Output Shape	Param #
vgg19 (Functional)	(None, 7, 7, 512)	20024384
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 50)	1254450
dropout (Dropout)	(None, 50)	0
dense_1 (Dense)	(None, 20)	1020
dropout_1 (Dropout)	(None, 20)	0
dense_2 (Dense)	(None, 3)	63

Figura 29 – Imagen de la parte de clasificación del modelo con arquitectura VGG19. Se describen cada una de sus capas.

Se visualizan la cantidad total de parámetros de visión y clasificación, detallando la cantidad de aquellos entrenables correspondientes a clasificación y los no entrenables pertenecientes a visión.

```

Total params: 21279917 (81.18 MB)
Trainable params: 1255533 (4.79 MB)
Non-trainable params: 20024384 (76.39 MB)

```

Figura 30 – Imagen de los parámetros de la parte de clasificación del modelo de arquitectura VGG19.

Para el entrenamiento se eligió como en los casos anteriores un batch size de 16, y 25 épocas. También se realiza mediante las imágenes de train (train_X y train_Y), y se valida utilizando valid (valid_X y valid_Y). Durante el proceso, se detalla la exactitud y pérdida en entrenamiento y en validación.

El algoritmo de Early Stopping logra que luego de 5 valores bajos luego del más alto detectado de val_accuracy el entrenamiento finalice. Es así, que el entrenamiento llegó a las 10 épocas:

```

Epoch 1/25
375/375 [=====] - 46s 101ms/step - loss: 0.2563 - accuracy: 0.9683 - val_loss: 0.4330 - val_accuracy: 0.9733
Epoch 2/25
375/375 [=====] - 39s 105ms/step - loss: 0.1143 - accuracy: 0.9890 - val_loss: 0.1509 - val_accuracy: 0.9867
Epoch 3/25
375/375 [=====] - 38s 100ms/step - loss: 0.0722 - accuracy: 0.9930 - val_loss: 0.0941 - val_accuracy: 0.9900
Epoch 4/25
375/375 [=====] - 35s 93ms/step - loss: 0.0810 - accuracy: 0.9925 - val_loss: 0.3713 - val_accuracy: 0.9767
Epoch 5/25
375/375 [=====] - 35s 94ms/step - loss: 0.0689 - accuracy: 0.9952 - val_loss: 0.0557 - val_accuracy: 0.9950
Epoch 6/25
375/375 [=====] - 35s 94ms/step - loss: 0.0539 - accuracy: 0.9970 - val_loss: 0.1696 - val_accuracy: 0.9950
Epoch 7/25
375/375 [=====] - 37s 98ms/step - loss: 0.0488 - accuracy: 0.9957 - val_loss: 0.4822 - val_accuracy: 0.9583
Epoch 8/25
375/375 [=====] - 37s 99ms/step - loss: 0.0171 - accuracy: 0.9987 - val_loss: 0.2891 - val_accuracy: 0.9817
Epoch 9/25
375/375 [=====] - 37s 99ms/step - loss: 0.1083 - accuracy: 0.9973 - val_loss: 0.3721 - val_accuracy: 0.9733
Epoch 10/25
375/375 [=====] - 36s 95ms/step - loss: 0.0138 - accuracy: 0.9988 - val_loss: 0.1270 - val_accuracy: 0.9850

```

Figura 31 – Imagen del entrenamiento de la CNN con arquitectura VGG19.

Luego de este entrenamiento se realizó la evaluación del modelo, esto se logra mediante evaluate utilizando al igual que antes test (test_X y test_Y). Se llegó a un accuracy de 99 % y loss de 0.0454.

```

19/19 [=====] - 16s 475ms/step - loss: 0.0454 - accuracy: 0.9983
Test loss: 0.045437537133693695
Test accuracy: 0.9983333349227905

```

Figura 32 – Evaluación de la CNN con arquitectura VGG19 luego de su entrenamiento.

Seguidamente, se realizaron gráficas para mostrar de manera visual el proceso de entrenamiento.

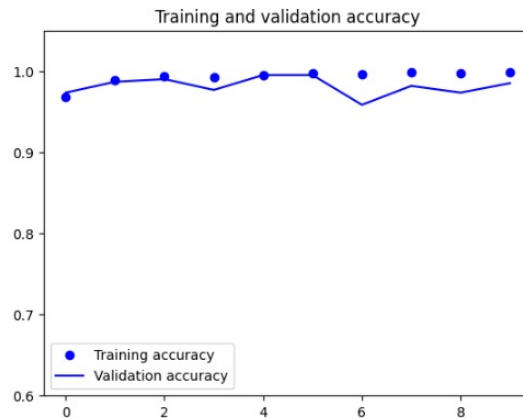


Figura 33 – Gráfico de exactitud en el proceso de entrenamiento y validación de CNN con arquitectura VGG19 (training and validation accuracy).

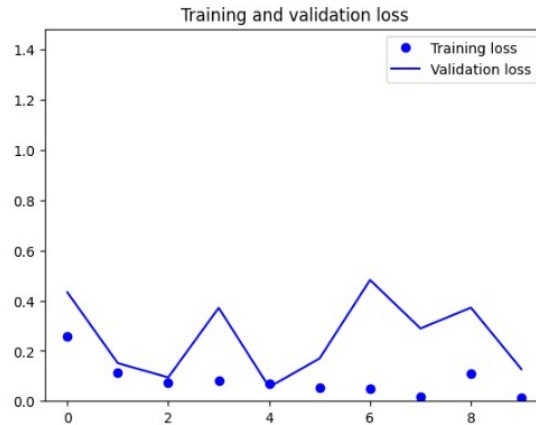


Figura 34 – Gráfico de pérdida en el proceso de entrenamiento y validación de CNN con arquitectura VGG19 (training and validation loss).

Se realizó una tabla con `classification_report`. Se detallan en las columnas precision, recall y f1-score de cada una de las tres clases. Asimismo, se destaca nuevamente la exactitud redondeada de 100 % del modelo, así como macro y weighted avg con los mismos valores para cada uno de los parámetros de las columnas previamente mencionadas.

	precision	recall	f1-score	support
Class 0	1.00	1.00	1.00	200
Class 1	1.00	1.00	1.00	200
Class 2	1.00	0.99	1.00	200
accuracy			1.00	600
macro avg	1.00	1.00	1.00	600
weighted avg	1.00	1.00	1.00	600

Figura 35 – Tabla de `classification_report` de CNN con arquitectura VGG19.

Finalmente se efectuó una Matriz de Confusión. El análisis de esta indica que nuevamente como en VGG16 las imágenes de edificios, bosques y glaciares se predicen con alta precisión. Esto se evidencia en la diagonal de la matriz, donde hay números cercanos a 200 mostrando que acierta la mayor parte de los datos. Así, queda demostrando que el modelo VGG19 clasifica correctamente la mayoría de las categorías. En definitiva, la diagonal en la matriz de confusión reafirma la capacidad del modelo para distinguir eficazmente entre las distintas imágenes.

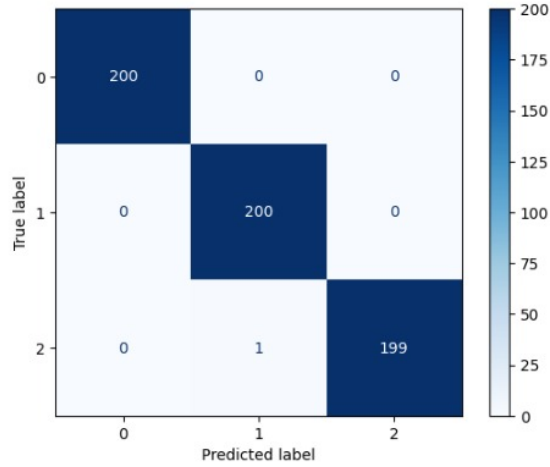


Figura 36 – Matriz de confusión VGG19.

4. Conclusiones

Habiendo realizado los tres modelos con arquitectura personalizada, VGG16 y VGG19, podemos comparar los siguientes parámetros:

Modelos	Accuracy	Loss
Personalizado	89.9 %	0.4734
VGG16	99.6 %	0.0525
VGG19	99.8 %	0.0454

Cuadro 1 – Cuadro comparativo de los modelos con arquitectura personalizada, VGG16 y VGG19

En conclusión, el trabajo demuestra la gran capacidad de los modelos VGG16 y VGG19 para realizar la clasificación en tres categorías de las imágenes del dataset propuesto, en comparación al modelo personalizado, el cual posee menor grado de exactitud. Uno de los aprendizajes más significativos fue en definitiva la eficacia de utilizar Transfer Learning para mejorar el rendimiento de los modelos.

Durante el desarrollo, se destacó la necesidad de un preprocesamiento de imágenes adecuado y técnicas de aumento de datos (Data Augmentation) para optimizar la performance del modelo. Asimismo, destacamos la importancia de tener un dataset balanceado, debido a que en anteriores experimentos concluimos que la falla de esto conlleva a resultados erróneos no deseados.

Además, la observación de la matriz de confusión permitió discernir los valores correctos y la identificación de errores con falsos negativos y falsos positivos, facilitando una mejor comprensión de los resultados de los modelos.

5. Referencias

- [1] Bonilla Carrión, C. (2020). Redes convolucionales.
- [2] Artola Moreno, Á. (2019). Clasificación de imágenes usando redes neuronales convolucionales en Python.
- [3] Cifuentes, A., Mendoza, E., Lizcano, M., Santrich, A., & Moreno-Trillos, S. (2019). Desarrollo de una red neuronal convolucional para reconocer patrones en imágenes. *Investigación y desarrollo en TIC*, 10(2), 7-17.
- [4] Ferro, G. Investigación en redes convolucionales y una aplicación a la detección de Peatones.
- [5] Enciso Ortiz, S. E. (2024). Determinación de la mejor Arquitectura de Redes Neuronales Convolucionales: VGG16, ResNet50 ó MobileNet para detección de la Neumonía 2023.
- [6] Sánchez-Alonso, D. (2022). Aplicación móvil para evitar fraudes en criptomonedas mediante la detección de logos (Master's thesis).
- [7] Irmak, E. (2020, November). A novel deep convolutional neural network model for COVID-19 disease detection. In 2020 Medical Technologies Congress (TIPTEKNO) (pp. 1-4). IEEE.)
- [8] De Luca, A. M., Irigoitia, M. E., Pérez, G. A., & Pons, C. F. (2021). Uso de la Técnica de Transfer learning en machine learning para la clasificación de productos en el banco alimentario de la plata. In IX Congreso Nacional de Ingeniería Informática/Sistemas de Información (CoNaIISI)(Mendoza, 4 y 5 de noviembre de 2021).